Lab 4: Tiled Convolution

1. Objective

The purpose of this lab is to get you more familiar with shared memory tiling techniques, handling complex boundary conditions, and using constant memory.

2. Preliminary work

Step 1: Log in to the GPU side of Roihu (see Lab 1 if the login is refused because your SSH certificate has expired):

```
ssh <username>@roihu-gpu.csc.fi
```

Step 2: If you have not already cloned the lab repository in Lab 1, do it now, in the directory where you would like to put it:

```
git clone https://github.com/jolilius/GP26_Labs.git
cd GP26_Labs
```

You are now ready to begin the lab.

3. Procedure

Step 1: Update your local repository to obtain the code needed for the assignment.

```
cd GP26_Labs
git pull
cd Lab_4
```

Step 2: Edit the file main.cu to add the constant memory copy and set the block and grid dimensions correctly.

Step 3: Edit kernel.cu to implement the shared memory tiled convolution. Use zero padding if needed.

Step 4: Compile and test your code. Programs that use a GPU run on a compute node in a Slurm job, so you submit `run.sh`, which builds the program on the node and runs it. Use the CSC project you joined in Lab 1 as the account:

```
sbatch --account=<your-project> run.sh          # Uses the default input image size
sbatch --account=<your-project> run.sh <m>      # Uses a square m x m input image
sbatch --account=<your-project> run.sh <m> <n>  # Uses an (m x n) input image matrix
```

The output is written to `lab4_<jobnumber>.out` and to `output.txt` (each job overwrites `output.txt`). Wait for the job to finish (`squeue --me` shows your jobs) and then read the file. A correct solution prints something like the following (the timings will differ):

```
Setting up the problem...0.009086 s
```

```
    Image: 600 x 1000
    Mask: 5 x 5
Allocating device variables...0.678557 s
Copying data from host to device...0.000173 s
Launching kernel...0.000055 s
Copying data from device to host...0.000184 s
Verifying results...TEST PASSED
```

Compile errors show up in `lab4_<jobnumber>.out`. Your code is expected to work for image sizes that are not divisible by your tile size, so test such sizes too. For the questions in step 5 you need the timings of several image sizes: keep the `lab4_<jobnumber>.out` files of those runs, and run the default size last so that `output.txt` holds the output for the default size.

Step 5: Answer the following questions in a new file named answers.txt:

1. What is the floating-point computation rate for the GPU kernel in this application? How does it scale with the size of the input image? To answer this question, try multiple sized inputs and calculate the rate for each using the timing measurements provided in the code. Make sure to justify your choice of input sizes.
2. What percentage of time is spent as overhead for using the GPU? Consider as overhead: device memory allocation time and memory copy time to and from the device. Do not include problem setup time or result verification time in your calculations of overhead or total execution time. Try this with multiple input sizes and explain how the overhead scales with the size of your input?

Step 6: Submit your assignment. You need to submit the following files:

- main.cu
- kernel.cu
- answers.txt
- output.txt (the output of your run, which must show `TEST PASSED`)

Compress the files and name them after your student name and student id like so:

```
Lab4_Name_id.tar
```

Upload the file to the assignment in Moodle.

4. Grading

Your submission will be graded based on the following criteria.

- Functionality/knowledge: 65%
  - Correct code and output results (`output.txt` shows `TEST PASSED`)
  - Correct usage shared and constant memory to cover global memory access latency
  - Correct handling of boundary cases
- Answers to question: 35%

- Correct answer to questions in step 5
- Sufficient work shown
- Neatness and clarity